UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS PROGRAMA DE INGENIERÍA DE SISTEMAS

GUÍA PRÁCTICA SEMANA 07 – 2° PARTE

Asignatura: Desarrollo de Aplicaciones Web (IS093A)
Unidad I: Desarrollo Web Frontend
Tema: React Hooks: useState, useEffect, useContext, useRef, useReducer, useCallback, useMemo y Hooks Personalizados

Alumno: **PARIONA RAMOS SAUL ANIBAL**

OBJETIVO DE LA PRÁCTICA

Debe desarrolla un **Dashboard de Gestión con Estado Complejo** que cumpla:

1. useReducer para estado de lista (add, remove, filter, sort) + useState para UI local.
2. useContext para tema claro/oscuro o preferencias de usuario.
3. useEffect para sincronizar preferencias con localStorage y cleanup.
4. useMemo para lista filtrada/ordenada, useCallback para handlers de botones.
5. useRef para auto-focus en input o acceso a nodo DOM sin re-render.
6. Hook personalizado useLocalStorage o useDebounce reutilizable.

Roles Técnicos

Rol	**Responsabilidad**
Arquitecto de Estado	useReducer, acciones, reducer function, inmutabilidad
Ingeniero de Efectos & Contexto	useEffect, useContext, sync con storage, cleanup
Optimizador de Rendimiento	useMemo, useCallback, useRef, análisis Profiler
QA & Hook Validator	Reglas de Hooks, ESLint warnings, documentación técnica, custom hook

Entregable 1: En su repositorio de Github

- Repositorio GitHub o ZIP con: package.json, src/, README.md
- README.md debe incluir:
  - Diagrama de flujo de estado y ciclo de render
  - Justificación técnica de cada hook utilizado (¿por qué no useState simple?)

REPOSITOTIO CON EL REDMI PEDIDO

https://github.com/saulPariona/Semana-7-Aguilar-parte-2.git

1. Diagrama de flujo de estado y ciclo de render

```
<App />
└── <ThemeProvider> (ThemeContext.tsx)
    ├── useLocalStorage('dashboard_theme', 'light')
    ├── useEffect: aplica la clase del tema al <body> y ejecuta cleanup
    └── useCallback: estabiliza toggleTheme()
        |
        ▼
    <Dashboard> (App.tsx)
    ├── useContext(ThemeContext): consume el tema global
    ├── useLocalStorage('dashboard_tasks'): persiste la lista
    ├── useReducer(tasksReducer, initialTaskState)
    |   ├── Estado: { items: [], filter: 'all', sort: 'asc' }
    |   └── Acciones: ADD, REMOVE, TOGGLE, FILTER y SORT
    ├── useCallback: estabiliza las acciones de tareas
    └── useState(showTaskView): simula desmontaje y cleanup
        |
        ▼
    <TaskList> (TaskList.tsx)
    ├── useState: controla el texto del formulario
    ├── useRef: conserva referencias y aplica auto-focus
    ├── useEffect: enfoca el input y libera el timer al desmontar
    ├── useMemo: filtra y ordena la lista procesada
    └── useCallback: estabiliza el envío del formulario
        |
        ▼
    <TaskItem>
    └── recibe la tarea y handlers mediante props
```

El flujo es unidireccional: el usuario genera una acción, el reducer produce un nuevo estado, `App` persiste los datos y `TaskList` recibe las props actualizadas para renderizar la vista correspondiente.

2. Justificación técnica de cada hook utilizado

Entregable 2: Validación Final

1. Ejecutar npm run dev y verificar cero warnings de react-hooks/exhaustivedeps en ESLint.

Cero Errores npx oxlint

```
PS C:\DAW\SEMANA7_PARTE2_AGUILAR\Semana7_Aguilar_Parte2> npx oxlint
Found 0 warnings and 0 errors.
Finished in 13ms on 9 files with 116 rules using 12 threads.
PS C:\DAW\SEMANA7_PARTE2_AGUILAR\Semana7_Aguilar_Parte2>
```

npm run dev

```
Finished in 13ms on 9 files with 116 rules using 12 threads.
PS C:\DAW\SEMANA7_PARTE2_AGUILAR\Semana7_Aguilar_Parte2> npm run dev

> semana7_aguilar_parte2@0.0.0 dev
> vite

Port 5173 is in use, trying another one...

  VITE v8.3.4  ready in 254 ms

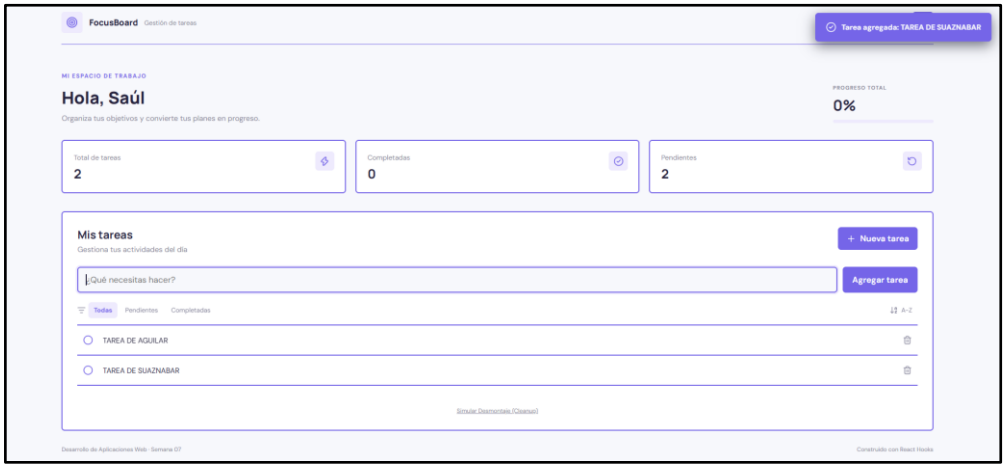
  ➜  Local:   http://localhost:5174/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

http://localhost:5174/



2. Abrir React DevTools Profiler: grabar interacción, identificar componentes que se re-renderizan y validar que useMemo/useCallback reducen commits.

AGREGANDO TAREAS Y NOTIFICACION




```tsx
const notify = useCallback((message: string) => {
  setAlertMessage(message)
  window.setTimeout(() => setAlertMessage(''), 2600)
}, [])
```

```tsx
const addTask = useCallback((title: string) => {
  dispatch({ type: 'ADD', title })
  notify(`Tarea agregada: ${title}`)
}, [notify])

const removeTask = useCallback((id: string) => {
  dispatch({ type: 'REMOVE', id })
  notify('Tarea eliminada correctamente')
}, [notify])
```

FILTRADO




```tsx
}, [])

const completedCount = useMemo(
  () => state.items.filter((task) => task.completed).length,
  [state.items],
)
```

```
const processedTasks = useMemo(() => [...tasks]
  .filter((task) => filter === 'all' || (filter === 'completed' ? task.completed : !ta
  .sort((a, b) => sort === 'asc' ? a.title.localeCompare(b.title) : b.title.localeComp

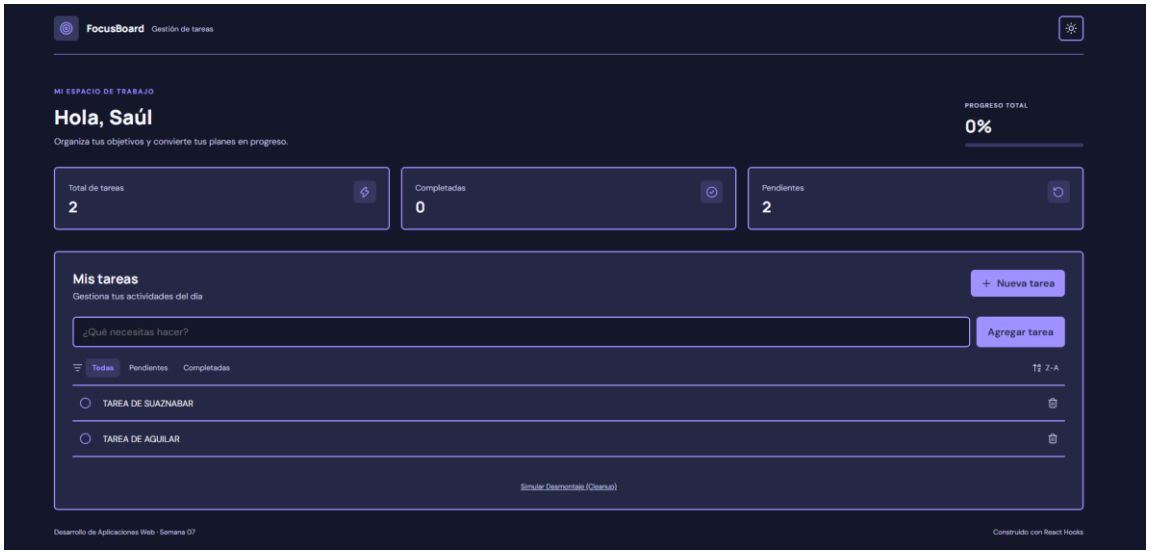
useEffect(() => {
```

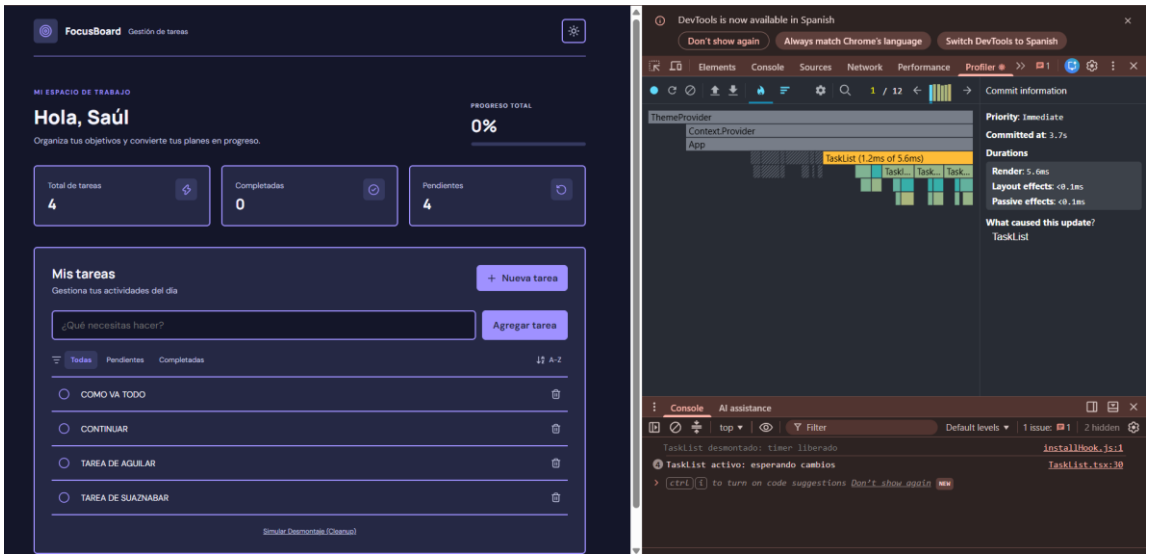
THEMA DE DIA




```
const toggleTheme = useCallback(() => setTheme((current) => current === 'light' ? 'dark' : 'light'), [setTheme])
return <ThemeContext.Provider value={{ theme, toggleTheme }}>{children}</ThemeContext.Provider>
```

THEMA DE NOCHE





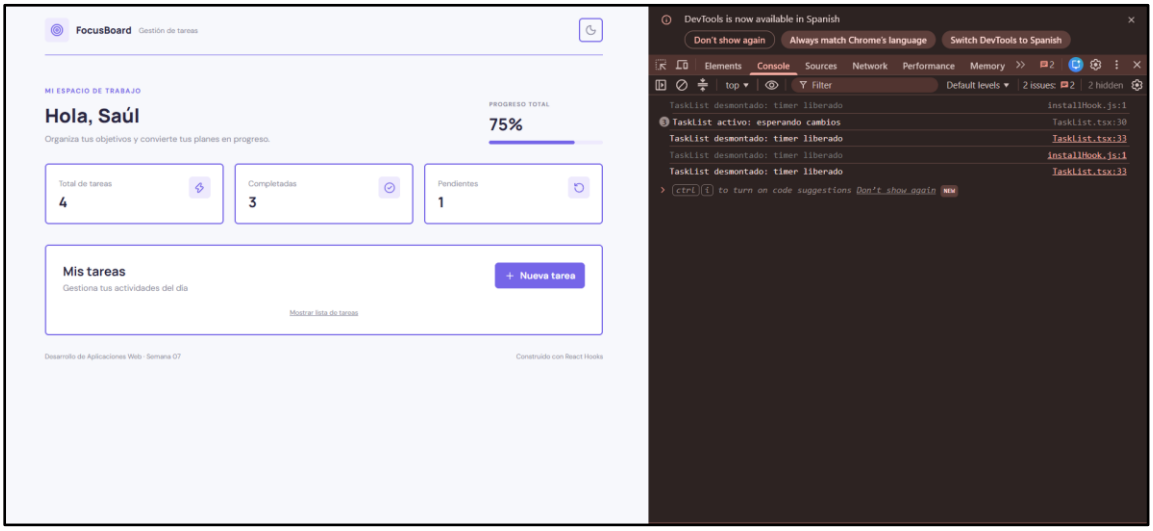
1. Validación de Rendimiento con React DevTools Profiler (useMemo y useCallback) Al grabar una interacción en la pestaña *Profiler* de React DevTools, se observa en el gráfico de flama (*Flamegraph*) que el componente TaskList se resalta en amarillo, indicando que se actualizó como respuesta a una interacción (tardando 1.2ms). Sin embargo, los rectángulos grises y rayados debajo de él representan a los componentes hijos (los ítems de las tareas), los cuales no se re-renderizaron. Esto evidencia que el uso estratégico de useCallback para estabilizar los *handlers* y la memoización de la lista filtrada con useMemo evitan *renders* innecesarios, reduciendo eficazmente los *commits*.

2. Validación del Cleanup en useEffect (Prevención de Memory Leaks) En la pestaña *Console* inferior, se registra el mensaje "TaskList desmontado: timer liberado". Esto demuestra que al simular la navegación rápida o utilizar la opción *"Simular Desmontaje (Cleanup)"* (ubicada en la parte inferior de la interfaz web), el componente se destruye correctamente y React ejecuta la función de limpieza (return () => { ... }) del useEffect, liberando recursos de monitoreo y evitando fugas de memoria (*memory leaks*).

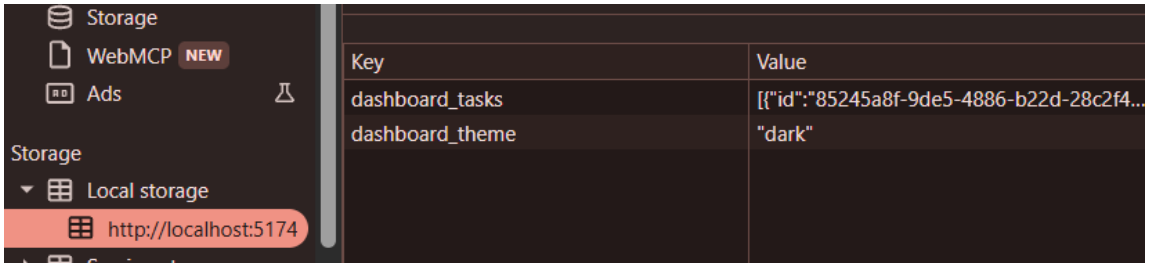
3. Estado Complejo (useReducer) y Contexto Global (useContext) La interfaz del Dashboard *"FocusBoard"* confirma la actualización reactiva del estado complejo manejado por useReducer, reflejando correctamente la contabilidad de tareas (4 totales, 0 completadas, 4 pendientes). Asimismo, la visualización en modo oscuro nativo confirma el correcto consumo de las preferencias del usuario a través de useContext.

3. Verificar que useEffect tenga cleanup activo (simular navegación rápida o desmontaje).

SIMULADO DE DESMONTAJE USANDO CONSOLES, VERIFICANDO EL useEffect



THEMA GUARDADO EN LOCAL STORAGE



4. Evidenciar y explicar con sus propias palabras cada una de las validaciones con capturas de pantalla.

LAS CAPTURAS YA SE REALIZARON EN LOS APARTADOS SUPERIORES

EXPLICACION

Este entregable presenta un repositorio en GitHub completamente estructurado con package.json, src/ y un archivo README.md detallado que incluye el diagrama de flujo del ciclo de renderizado, además de la justificación técnica para sustituir useState simple por useReducer en la gestión de un estado complejo con lógica centralizada. El proyecto cuenta con persistencia en localStorage, un contexto global mediante useContext para el cambio de tema claro/oscuro y supera las pruebas iniciales de validación al ejecutar npm run dev y el linter npx oxlint sin generar ninguna advertencia sobre dependencias (react-hooks/exhaustivedeps) ni errores en el código.

Las validaciones técnicas demuestran de forma práctica la optimización y estabilidad de la aplicación mediante el uso avanzado de React Hooks. Al analizar las capturas con React DevTools Profiler, se corrobora que la implementación de useMemo y useCallback evita renders innecesarios en los componentes hijos (TaskItem) reduciendo de forma drástica los commits del navegador. Finalmente, las herramientas de diagnóstico confirman el correcto funcionamiento de las funciones de limpieza (cleanup) en los useEffect, liberando temporizadores al desmontar el componente y previniendo fugas de memoria (memory leaks), mientras que la consola de desarrollo evidencia la persistencia local de los datos.

5. Subir a GitHub así mismo subir en rar su proyecto en el ADESA.

Rúbrica de Evaluación

Criterio	**Peso**
Uso correcto de useReducer + manejo de estado complejo	20%
useContext + useEffect (sync, cleanup, dependencias)	20%
useMemo/useCallback aplicados estratégicamente (sin overuse)	20%
useRef + Hook personalizado funcional y reutilizable	20%
Validación Profiler/ESLint, documentación	20%

CÓDIGO BASE DE LA GUÍA PRÁCTICA (Refactorizar) src/contexts/ThemeContext.jsx

```jsx
import { createContext, useState, useEffect } from 'react'

export const ThemeContext = createContext()

export const ThemeProvider = ({ children }) => {
  // TODO: Inicializar estado con localStorage o valor por defecto
  const [theme, setTheme] = useState('light')

  // TODO: useEffect para aplicar clase al <body> y persistir en localStorage
  // TODO: Función toggleTheme expuesta via value

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  )
}
```

src/hooks/useTasksReducer.js

```js
// TODO: Definir tipos de acción: ADD, REMOVE, FILTER, SORT
const initialState = { items: [], filter: 'all', sort: 'asc' }

function tasksReducer(state, action) {
  switch (action.type) {
    // TODO: Implementar casos inmutables
    default: return state
  }
}

export default tasksReducer
```

src/hooks/useLocalStorage.js (Hook Personalizado)

```js
import { useState, useEffect } from 'react'

// TODO: Implementar hook que sincronice estado con localStorage
// Recibe key y defaultValue, retorna [value, setValue]
export const useLocalStorage = (key, defaultValue) => {
  // ...
}
```

src/components/TaskList.jsx

```jsx
import { useMemo, useCallback, useRef } from 'react'

const TaskList = ({ tasks, onRemove, onToggle }) => {
  const inputRef = useRef(null)

  // TODO: useMemo para filtrar/ordenar tareas sin recalcular en cada render
  const processedTasks = useMemo(() => {
    // ...
  }, [tasks])

  // TODO: useCallback para estabilizar handlers pasados a hijos
  const handleRemove = useCallback((id) => {
    onRemove(id)
  }, [onRemove])

  // TODO: useEffect para focus al input cuando se añade tarea
  // ...

  return (
    <ul>
      {processedTasks.map(task => (
        <li key={task.id}>
          <span>{task.title}</span>
          <button onClick={() => handleRemove(task.id)}>Eliminar</button>
        </li>
      ))}
      <input ref={inputRef} placeholder="Nueva tarea..." />
    </ul>
  )
}

export default TaskList
```

src/App.jsx

```jsx
import { useReducer, useContext } from 'react'
import tasksReducer from './hooks/useTasksReducer'
import { ThemeProvider, ThemeContext } from './contexts/ThemeContext'
import TaskList from './components/TaskList'

const Dashboard = () => {
  const [state, dispatch] = useReducer(tasksReducer, { items: [], filter: 'all', sort: 'asc' })
  const { theme, toggleTheme } = useContext(ThemeContext)

  // TODO: Conectar dispatch a handlers, renderizar TaskList y controles
  return (
    <div className={`dashboard ${theme}`}>
      <button onClick={toggleTheme}>Cambiar Tema</button>
      <h1>Gestor de Tareas</h1>
      {/* TODO: Renderizar lista y formularios con dispatch */}
    </div>
  )
}

export default function App() {
  return (
    <ThemeProvider>
      <Dashboard />
    </ThemeProvider>
  )
}
```

NOTAS TÉCNICAS CLAVE

Concepto	**Regla / Aplicación**
Reglas de Hooks	Solo en nivel superior, nunca en loops/condiciones/funciones anidadas. Solo en componentes funcionales o custom hooks.
useEffect	Array de dependencias determina cuándo se ejecuta. Cleanup (return () => {}) evita memory leaks y race conditions.
useContext	Ideal para evitar prop drilling, pero no para estado que cambia frecuentemente (dispara re-render en todos los consumidores).

useReducer	Preferible a useState cuando el estado es complejo, tiene múltiples sub-valores o la lógica de actualización depende del estado anterior.

useMemo / useCallback	Solo optimizar si hay bottleneck medible. Memoizar objetos/funciones pasados como props a hijos React.memo.
useRef	Persiste entre renders sin disparar re-render. Ideal para acceso DOM, timers, o valores mutables que no afectan UI.
Profiler	React DevTools → Profiler → Record. Analiza render time, commits, why did this render?. No optimizar sin medir.